

Canopy

Audit Report

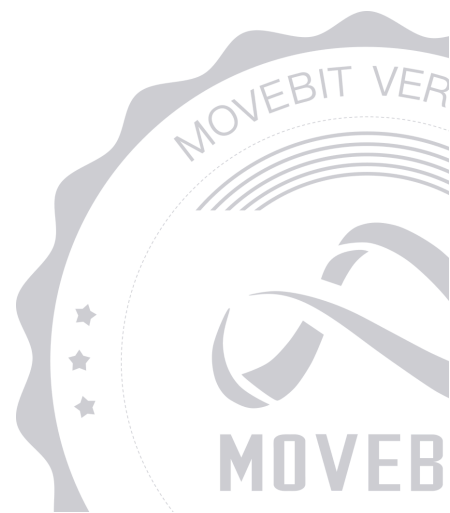


contact@bitslab.xyz



https://twitter.com/movebit_

Tue Jun 11 2024



Canopy Audit Report

1 Executive Summary

1.1 Project Information

Description	An asset management project
Type	DeFi
Auditors	MoveBit
Timeline	Tue Mar 26 2024 - Tue Jun 11 2024
Languages	Move
Platform	Aptos
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/pontem-network/ichi-vaults
Commits	4518e2ea02a99ef01cdba62dbfd5f467bde4be88e7b17c6976c2310ac17ab4cd97d5783e4720fc16 4518e2ea02a99ef01cdba62dbfd5f467bde4be88cd1a53fcb94ff6cd0d2b62124c6dca5ad6a078b3 4518e2ea02a99ef01cdba62dbfd5f467bde4be888edfaa876ab88780564a22333209d4f00b36d6db 4518e2ea02a99ef01cdba62dbfd5f467bde4be885f7ed6cf12d5fd3a33c10326b3c7b47e66768784 15bb9af28ba5d544207a745cf8fb46bad18a6c23

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
MOV	Move.toml	94c9a7f4d6486de5ad88469d2414 ec1c70dacc9a
ENT	sources/entry.move	589a4330a28f23ff6c469ee6f65c44 d84154b217
ITO	sources/ichi_token.move	d106b6e82826e8e47dac1ea9ac72 5e6c61240fa6
CON	sources/config.move	20a97e684406e5db885b5acae74f 7fae9c0c1a3b
VAU	sources/vault.move	175e1774807a8dc637dceab530bd f083df0837c6
ENT	sources/entry.move	7db6872b169859f33b677ec15eea 4fe3b3e325db
OLI	sources/oracle_library.move	6385cea3f50bd9969217d00b2858 1a482b2b0a55
MOV	Move.toml	df2df6a631549a8376ad51eb0af59 e45d3cb2144
CON	sources/config.move	1e584b5bc0420f163805c8627f1d7 e96ee4ee3c3
VAU	sources/vault.move	eed6e6367f3e420c97768ad9aea02 a5316e67f28

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	5	5	0
Informational	1	1	0
Minor	2	2	0
Medium	0	0	0
Major	2	2	0
Critical	0	0	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the "**Testing and Automated Analysis**", "**Code Review**" and "**Formal Verification**" strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by [Pontem](#) to identify any potential issues and vulnerabilities in the source code of the [Canopy Ichi Vault](#) smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 5 issues of varying severity, listed below.

ID	Title	Severity	Status
CON-1	Lack of Events Emit	Minor	Fixed
VAU-1	Check the Implicit Correspondence between lb_token and Bin	Major	Fixed
VAU-2	Underutilization of Funds when Providing Liquidity	Major	Fixed
VAU-3	Lack of Checking Coin Balances Leads to Redundancy	Minor	Fixed
VAU-4	If Statement Is Always True	Informational	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the [Canopy Ichi Vault](#) Smart Contract :

Admin

- The Admin can initialize the vault module through `initialize` .
- The Admin can set the base fee of the protocol through `set_base_fee()` .
- The Admin can set the affiliate fee through `set_split_fee()` .

Vault Owner

- The Vault Owner can create a new vault through `create_vault<X, Y, BinStep>()` .
- The Vault Owner can rebalance the vault through `rebalance<X, Y, BinStep>()` .
- The Vault Owner can set the maximum coins amount that is deposited into the vault through `set_max_deposits<X, Y, BinStep>()` .
- The Vault Owner can set the affiliate account address of the vault through `set_affiliate<X, Y, BinStep>()` .
- The Vault Owner can set the maximum liquidity token supply of the vault through `set_max_total_supply<X, Y, BinStep>()` .

User

- The User can deposit coins into the vault through `deposit<X, Y, BinStep>()` .
- The User can burn the LP tokens and withdraw the deposited coins through `withdraw<X, Y, BinStep>()` .

Fee Recipienter

- The Fee Recipienter can claim the recipient fee through `claim_recipient_fee<X, Y, BinStep>()` .

Affiliate

- The Affiliate can claim the affiliate fee through `claim_affiliate_fee<X, Y, BinStep>()` .

4 Findings

CON-1 Lack of Events Emit

Severity: Minor

Status: Fixed

Code Location:

`sources/config.move#41,51`

Descriptions:

The smart contract lacks appropriate events for monitoring sensitive operations, which could make it difficult to track sensitive actions or detect potential issues.

Suggestion:

It is recommended to emit events for those sensitive functions.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

VAU-1 Check the Implicit Correspondence between lb_token and Bin

Severity: Major

Status: Fixed

Code Location:

sources/vault.move#508

Descriptions:

In the `posidon()` function, to calculate the assets corresponding to the shares in the swap pool, such a loop is used in this code

```
let length = vector::length(lb_tokens);  
let i = 0;  
while (i < length) {//...}
```

In fact, what the code wants to check is the asset value from `bin_lower` to `bin_upper` corresponding to `vault`. Since the length of `lb_token` implicitly corresponds to `bin_upper - bin_lower`, more specifically, `lb_token` Each share in `corresponds one-to-one` from `bin_lower` to `bin_upper`. Therefore, the length of `lb_token`` can be used as the number of loops.

In the `mint_liq()` function, the mapping check is not strictly performed, which can lead to serious consequences.

```
// Prevent ERR_ZERO_SHARES_AMOUNT error  
  
let price = bin_helper::get_price_from_id_fp64(i, bin_steps::get_bin_step<BinStep>());  
let liq_fp64 = bin_helper::get_liquidity_fp64(liquidity_x, liquidity_y, price);  
let liq = liq_fp64 / (1 << 64);  
  
if (liq > 0) {  
    vector::push_back(&mut vec_x, liquidity_x);  
    vector::push_back(&mut vec_y, liquidity_y);  
    vector::push_back(&mut bin_ids, i);
```

```
vector::push_back(&mut total_as_y, (liq as u64));  
};
```

An attacker can construct a `price` that is small enough so that `liq` is equal to 0. This will bypass the code in the if statement and skip investing in some `bin` where `price` is set too small and `liq` is 0. Thus Making the actual minted token range a subset of `bin_lower` to `bin_upper` will cause a calculation error in `position()`. Because in `position()` the code starts checking from `bin_lower`, And retrieve `bin` as long as `lb_token`. But at this time, the tokens in `lb_token` do not correspond to the correct `bin`. In this way, the assets calculated by the code always deviate from the actual value, and there is no way to know whether it is Too big or too small - it depends on the parameters in the actual `bin`.

Error calculations caused by the `position()` function will affect the `get_total_amounts()` function, which in turn affects the numerical calculation in `deposit()`.

Let's first assume that the calculated value of `position()` is too small, which will cause the share obtained by `deposit()` to be too large. An attacker may construct the following attack:

1. The attacker creates a `vault`
2. The victim user calls `deposit()` to pledge
3. The attacker calls `rebalance()` and sets extreme prices, causing the `positon()` function to not work properly.
4. The attacker calls `deposit()` to obtain more shares than the actual value
5. The attacker calls `withdraw()` to arbitrage and exit.

If the calculated value of `position()` is too large, this will cause the share obtained by `deposit()` to be too small.

1. Create `vault`
2. The attacker user calls `deposit()` to pledge
3. The attacker calls `rebalance()` and sets extreme prices, causing the `positon()` function to not work properly.
4. The user calls `deposit()` to obtain a smaller share than the actual value.
5. The attacker calls `withdraw()` to arbitrage and exit.

Suggestion:

It is recommended to add the following checks to the `mint_liq()` function:

```
assert!(liq!=0);
```

Regardless of the fix, if the `positon()` function maintains the original design, please check the implicit correspondence between `lb_token` and `bin_lower` , `bin_upper` .

Resolution:

This issue has been fixed. The client has adopted our suggestions.

VAU-2 Underutilization of Funds when Providing Liquidity

Severity: Major

Status: Fixed

Code Location:

`sources/vault.move#483`

Descriptions:

We consider a situation where the function `mint_liq()` is called to invest when

`active_bin < bin_lower < bin_upper` .

In `mint_liq()` , we will divide the funds equally, and the number of equal shares depends on

`range_left` and `range_right` .

Since `coin_y` does not participate in investment in the set situation, we will get the amount of `partition_x` . It is the difference between `bin_upper` and `active_bin`

```
let range_right = if (active_bin < bin_upper) { bin_upper - active_bin + active_bin_in_range }  
else { 1 };
```

```
let partition_x = if (range_right > 0) { val_x / (range_right as u64) } else { 0 };
```

But the price traversed is from `bin_lower` to `bin_upper`

```
let i = bin_lower;  
while (i <= bin_upper) {//...}
```

Therefore, we can draw a conclusion: when `bin_upper` is further away from `active_bin` and the closer the distance between `bin_upper` and `bin_lower` is, the actual investment amount is less and the fund utilization rate is lower. Under some investment settings, This can lead to severe underutilization of funds.

Suggestion:

It is recommended to make sure if this is in line with your design, if not please consider redesigning it.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

VAU-3 Lack of Checking Coin Balances Leads to Redundancy

Severity: Minor

Status: Fixed

Code Location:

sources/vault.move#510

Descriptions:

This `min_liq()` function is called as a relatively low-level function. When the upper-level function executes some complex business logic, a large number of unchecked values may be input. When some values are meaningless, we should return early to save unnecessary gas costs.

Suggestion:

```
let val_x = coin::value(&coin_x);
let val_y = coin::value(&coin_y);
if(val_x==0&&val_y==0)
{
    (vector[], vector[], coin_x, coin_y)
}
```

It is recommended to add check under codes above.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

VAU-4 If Statement Is Always True

Severity: Informational

Status: Fixed

Code Location:

sources/vault.move#478

Descriptions:

Since `range_right` and `range_left` are always greater than 0, this causes the if statement to always be true and meaningless.

```
let partition_x = if (range_right > 0) { val_x / (range_right as u64) } else { 0 };  
let partition_y = if (range_left > 0) { val_y / (range_left as u64) } else { 0 };
```

Suggestion:

It is recommended to delete the if statement.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

